

# DualGuard — 기술 질문 예상지

2026-09-03 · 심사위원 중 개발자가 있을 때 나올 만한 질문 · 모르면 "안 했습니다"가 정답이다  
값은 전부 코드와 검증 리포트에서 확인한 것이다.

기술 질문에 답하는 요령 — 파라미터를 외워 읊는 것보다 "왜 그 값인가"를 말하는 게 낫다. 대부분의 값은 실측으로 정했으므로 "○○를 재봤더니 이래서 이렇게 했습니다" 형태로 답하면 전부 커버된다. 근거가 없는 값은 솔직히 "기본값을 그대로 썼습니다"라고 말할 것.

## 1. STT · 오디오 전처리

"STT는 뭘 쓰고, 왜 그걸 골랐습니까?"

**faster-whisper**입니다. openai-whisper가 아니라 CTranslate2 백엔드 구현체인데, 같은 정확도에서 **CPU 추론이 4배 이상 빠릅니다**. 시연 환경에 GPU가 없을 가능성이 커서 그렇게 골랐습니다.

모델 크기는 **용도별로 다르게** 씁니다 — 파일 분석은 `small`, 실시간 스트리밍은 `base` 입니다. `tiny/base/small`을 CER·처리시간·화이트노이즈 강건성으로 비교해서 정한 값입니다.

"오디오 리샘플링은 어떻게 합니까?"

`scipy.signal.resample_poly`를 씁니다. 실제 통화는 전화망 8kHz거나 44.1kHz 녹음이라 거의 항상 리샘플링을 거치는데, **선형 보간은 저역통과 필터가 없어 에일리어싱이 생깁니다**. 그 인공물을 음성 판별기가 합성 흔적으로 오인할 수 있어서 다상 필터를 씁니다. `scipy`가 없으면 선형 보간으로 풀백합니다.

## 2. 화법 분류 (콘텐츠 엔진)

"LLM이 없으면 어떻게 동작합니까? 단순 키워드 매칭인가요?"

키워드만은 아닙니다. 풀백은 키워드 규칙 + 한국어 임베딩 의미 유사도를 결합한 분류기입니다. 두 방식의 성능 차이가 큼니다 — 같은 21건에서 순수 키워드는 **66.7%**, 결합은 **90.5%**입니다.

결합 방식은 카테고리별로 두 점수 중 **높은 쪽을** 택하되, **면책·경고 문장이면 키워드 점수까지 함께 억제**합니다. "저희는 비밀번호를 절대 요구하지 않습니다" 같은 정상 안내에서, 임베딩만 억제하고 키워드를 두면 결합 결과가 여전히 오탐이기 때문입니다.

"콘텐츠 위험도는 어떻게 하나의 숫자가 됩니까?"

$0.5 \times \text{최고 카테고리 점수} + 0.5 \times \text{상위 3개 평균}$ 입니다. 기획서 공식 그대로입니다.

최고점만 쓰면 한 카테고리가 튀는 것만으로 위험도가 올라가고, 평균만 쓰면 여러 기법이 동시에 쓰인 상황이 희석됩니다. 둘을 절반씩 섞은 형태입니다.

"LLM을 통화 전체가 아니라 구간마다 부르면 비용이 크지 않습니까?"

맞습니다. 통화 하나에 수십 번 호출됩니다. 그래도 구간 단위로 하는 이유는 결과 화면이 "언제 무엇이 걸렸는지" 타임라인을 요구하기 때문입니다. 비용은 이렇게 줄였습니다 — Claude는 **reasoning effort**를 **low**로 두고 시스템 프롬프트에 **캐시 breakpoint**를 걸었고, Gemini는 무료 티어 RPM 제한 (실측 5 RPM)에 걸리므로 호출 간격을 강제할 수 있게 했습니다. 실패해도 예외를 던지지 않고 오프라인 분류기로 풀백합니다. 구간 하나만 떼면 맥락이 없어서("네 알겠습니다"만으로는 판단 불가) **앞뒤 구간을 함께** 넘깁니다.

"LLM 출력이 JSON이 아니면 어떻게 합니까?"

백엔드마다 **구조화 출력을 강제**합니다 — Claude는 `structured outputs`, Gemini는 `response_schema`, OpenAI는 `response_format`, Ollama는 `format` 에 JSON 스키마를 넣습니다. 그래서 "JSON만 출력하세요"라고 프롬프트에 빌고 파싱 실패를 재시도로 때우는 코드가 없습니다.

다만 수치 범위 제약(**minimum/maximum**)은 어느 스키마도 보장하지 않아서, 0~100 범위 검증은 백엔드에 맡기지 않고 **모든 백엔드의 공통 경로에서 한 번 더** 합니다.

## 3. RAG

"임베딩 모델은 뭘 씁니까? 다국어 모델을 쓰면 안 됩니까?"

`jhgan/ko-sroberta-multitask`, 한국어 전용입니다. 다국어 모델을 써봤는데 **무관한 한국어 문장을 동일 문장보다 가깝게** 판정했습니다. 실측표를 코드 주석에 남겨놨습니다. 벡터DB는 ChromaDB, 거리는 코사인입니다.

"유사도 임계값은 어떻게 정했습니까?"

코사인 유사도 **0.55 이상만** 채택합니다(거리 0.45 이하). 한국어 문장 임베딩은 관련/무관 경계가 넓어서 그 사이 어디에 뒤도 되는 구간이 있는데, **보수적인 쪽으로** 잡았습니다.

채점은 유사도 절댓값이 아니라 **마진**으로 하고, **14자 미만 짧은 문장은 채점하지 않습니다** — 짧으면 임베딩이 불안정해서 아무 사례에나 붙습니다.

**"사례가 26건이면 RAG라고 부르기엔 작지 않습니까?"**

맞습니다. 26건에서 문서 **103개**를 만들어 색인합니다(사례 하나가 요약·대사 등 여러 문서로 쪼개집니다). 규모로 승부하는 게 아니라 **"왜 위험한지 근거를 보여주는 용도"**입니다. 확충 경로는 공공데이터포털의 사기 수법 통계로 잡아냈습니다.

## 4. 음성 위조 판별 (AASIST)

"AASIST는 4초 입력인데 긴 통화는 어떻게 처리합니까?"

원본 학습 길이가 **16kHz 64,600 샘플(약 4.04초)** 고정입니다. 통화는 훨씬 기니까 **슬라이딩 윈도우**로 잘라 각각 추론하고 집계합니다. 4초보다 짧은 조각은 패딩합니다.

집계는 **상위 k개 평균(topk\_mean)**입니다. 최댓값만 쓰면 윈도우 하나의 노이즈가 전체 판정을 흔들고, 단순 평균은 짧은 위조 구간을 희석합니다.

"배치 크기나 메모리 제약은요?"

배치는 **4를 넘기지 않습니다**. Whisper-AASIST-Xception-임베딩 모델을 순차로 쓰는데 여유 메모리가 6GB 아래로 떨어지면 `mkl_malloc` 실패나 네이티브 크래시가 납니다. 후자는 try/except로 못 잡아서 애초에 피해야 합니다. 그래서 파이프라인이 **단계마다 모델을 해제**합니다.

"음성 오탐 원인을 어떻게 규명했습니까?" ← 깊게 물어올 수 있다

가설을 하나씩 통제해 제거했습니다. 코덱(G.711  $\mu$ -law, G.726 ADPCM, AMR-NB, GSM 06.10, Speex), 전화망 대역 제한, 재인코딩, 편집 — **전부** 기각됐습니다.

남은 게 음량이었는데, **레벨을 통일해 비교**하니 걸렸습니다.

| 음원                      | -39dB | -30dB | -25dB | -20dB |
|-------------------------|-------|-------|-------|-------|
| ASVspoof 진짜 (학습 도메인)    | 0.8   | 0.4   | 0.2   | 0.3   |
| Zeroth 낭독 (도메인 밖·진짜 사람) | 16.9  | 81.1  | 98.9  | 100.0 |
| 실제 통화 녹취 (도메인 밖·진짜 사람)  | 45.4  | 98.1  | 100.0 | 100.0 |

즉 음량은 방아쇠일 뿐이고 원인은 **도메인 일반화 실패**입니다. 레벨 정규화로는 못 고칩니다 — 조용한 입력을 100으로 올릴 뿐입니다.

## 5. 딥페이크 판별 (FF++ Xception)

"프레임을 어떻게 뽑고 몇 장을 봅니까?"

초당 **1프레임**으로 뽑고 상한을 둡니다. 각 프레임에서 **얼굴을 검출해 크롭**한 뒤 Xception에 넣습니다. 집계는 음성과 같은 **상위 k개 평균**입니다.

"얼굴이 안 잡히면 어떻게 됩니까?"

점수를 내지 않고 예외를 던집니다. 이건 버그가 아니라 의도한 동작입니다 — FF++ Xception은 **얼굴 크롭으로 학습된 모델**이라 전체 프레임을 넣으면 의미 없는 값이 나옵니다. 얼굴이 없으면 "영상 판정 불가"로 표시하고 음성·화법만으로 판단합니다.

"얼굴 검출기는 뭘 씁니까? dlib은요?"

YuNet이 1순위, Haar cascade가 폴백입니다. 둘 다 OpenCV 내장이라 dlib을 설치하지 않았습니 — 시연 환경에서 빌드가 필요한 의존성을 늘리고 싶지 않았습니.

실측하면 정면은 YuNet 100% / Haar 99%로 비슷한데, **고개를 돌리면 Haar가 84.8%로 떨어집니다**. 그래서 YuNet을 앞에 뒀습니.

"TTA를 쓴다고 했는데 어떤 방식입니까?"

좌우 반전본을 함께 넣고 큰 쪽을 씁니다. 평균이 아니라 최댓값인 이유는, 한쪽이 깨끗하게 나오면 **평균이 신호를 희석**하기 때문입니다. 실측으로 정탐 +3%p, 오탐은 동일했습니다. 환경변수로 끌 수 있습니다.

"점수 재척도(calibration)는 왜 하고 어떻게 했습니까?"

원점수가 0 아니면 100으로 극단적으로 쏠려서 그대로 보여주면 과신을 부릅니다. 또 임계값을 여러 개 관리해야 해서, **로짓 공간 1차 변환**으로 재척도해 판정 경계가 50 하나가 되도록 맞췄습니.

중요한 건 순서입니다 — 프레임 점수는 원점수로 두고, **집계한 뒤 재척도**를 한 번만 적용합니다. 프레임마다 변환한 뒤 집계하면 적합 방식(영상 단위)과 어긋나 판정 경계가 50에서 32로 밀립니다. 실제로 겪은 문제입니다.

## 6. 통합 스코어링

### "audio와 video 점수는 어떻게 합칩니까?"

최대값입니다. 가중평균도 비교해봤는데, 4가지 조합(진짜/위조 × 음성/영상) 중 **2개를 놓쳤습니다**. 한쪽만 위조된 경우가 희석되기 때문입니다. 한계도 있습니다 — max라서 "영상만 위조"와 "둘 다 위조"의 미디어 점수가 똑같이 100입니다. 구분이 필요하다면 다른 결합을 써야 합니다. 문서에 적어놨습니다.

### "가중치와 임계값이 코드 여러 곳에 흩어져 있지 않습니까?"

한 곳씩만 있습니다. 가중치와 공식은 `fraud_risk_score.py` 의 `DEFAULT_STRATEGY · DEFAULT_CONTENT_WEIGHT`, 신호등 경계는 `pipeline.py` 의 `LEVEL_THRESHOLDS` 입니다. 다른 곳에 하드코딩하지 않는 것을 팀 규칙으로 문서에 적어놨습니다.

## 7. 실시간 스트리밍 아키텍처

### "실시간은 어떻게 3배속이 나오니까?"

모델을 세션 동안 메모리에 상주시킵니다. 예전에는 청크마다 `/api/analyze` 를 부르고 결과를 폴링했는데, 그러면 청크마다 모델을 다시 로드해서 5초 청크 하나에 30초 넘게 걸렸습니다. 지금은 세션 API로 바뀌서 **왕복이 한 번**이고, 오디오 78초를 26~29초에 처리합니다.

### "동시 사용자는 몇 명까지 됩니까?" ← 아픈 질문

지금 구조는 동시에 한 세션만 받습니다. 두 번째 요청은 409로 막습니다. 메모리 때문입니다 — 모델을 여러 벌 올리면 프로세스가 죽습니다. 추론도 워커 1개짜리 스레드 풀에 넘겨 직렬화합니다. 이걸 프로토타입의 한계이고 확장 방향은 명확합니다 — 추론을 별도 워커 프로세스로 분리하고 큐를 두면 수평 확장이 됩니다. 오늘 발표 범위에서는 하지 않았습니다.

여기서 변명하지 마라. "메모리 때문에 한 세션"이라고 말하고 확장 방향만 한 줄 붙이는 게 제일 깔끔하다.

### "추론이 5초보다 오래 걸리면 청크가 밀리지 않습니까?"

밀립니다. 그래서 처리 중이면 새 청크를 버립니다. 큐에 쌓지 않습니다. 실시간 경고에서는 오래된 결과를 늦게 보여주는 것보다 최신 상태를 보는 게 낫기 때문입니다. 버린 청크 수는 화면에 표시됩니다.

### "브라우저가 보내는 청크를 서버가 어떻게 디코딩합니까?" ← 실제로 고생한 부분

여기서 실제로 버그를 겪었습니다. `MediaRecorder` 에 `timeslice`를 주면 **WebM(EBML) 헤더가 첫 청크에만** 실립니다. 두 번째부터는 헤더 없는 Cluster 데이터만 와서 독립적으로 디코딩이 안 됩니다. 그래서 첫 청크에서 초기화 세그먼트를 캐싱해두고, 헤더 없는 후속 청크 앞에 붙여서 복원합니다. Segment 크기가 "unknown/streaming"이라 이어붙이면 유효하게 파싱됩니다. 클라이언트에서도 무음 청크를 버리는 필터가 헤더가 든 첫 청크까지 버리는 문제가 있어서, 첫 청크는 크기와 무관하게 보내도록 고쳤습니다.

### "세션이 안 닫히면 어떻게 됩니까?"

청크가 90초간 끊기면 죽은 세션으로 보고 자동 정리합니다. 브라우저가 종료 요청을 못 보내고 죽는 경우가 실제로 있어서 넣었습니다.

## 8. 프론트엔드 · 브라우저

### "차트 라이브러리를 왜 안 썼습니까?"

외부 의존성을 하나도 안 씁니다. 차트도 SVG를 직접 그립니다. 시연 환경의 네트워크를 믿을 수 없어서입니다 — CDN이 막히면 화면이 통째로 죽습니다. 번들링을 하면 빌드 단계가 생기는데 그것도 시연 직전에 깨질 수 있는 요소입니다.

### "통화 화면 위에 경고를 띄운다는 게 기술적으로 어떻게 가능합니까?"

**Document Picture-in-Picture** API(크롬 116+)를 씁니다. 웹페이지는 보안상 다른 사이트 위에 아무것도 그릴 수 없습니다 — 그게 가능하면 피싱 도구가 되니까요. PiP는 브라우저가 정식으로 허용한 별도 창이고, 브라우저 밖 다른 프로그램 위에도 뜹니다. 지원하지 않는 브라우저면 조용히 넘어가고 기존처럼 대시보드 탭에서 봅니다.

### "확장 프로그램도 만들었다고 했는데 어떻게 검증했습니까?"

실제 크롬을 띄워 CDP로 자동 검증합니다. --load-extension 이 Chrome 137+에서 막혀서 Extensions.loadUnpacked 를 씁니다.  
까다로웠던 건 tabCapture 의 activeTab 권한입니다. CDP로 만든 가짜 제스처로는 절대 안 떨어져서, 확장 단축키를 OS 레벨로 실제로 눌러 정식으로 부여받습니다. 우회가 아니라 진짜 사용자 호출입니다.

## 9. 평가 방법론 · 재현성

"대용량 공개 데이터셋을 어떻게 다 받았습니까?"

다 받지 않았습니다. HTTP Range 요청으로 원격 파일을 로컬 파일처럼 읽는 래퍼를 만들어서, zipfile 과 pyarrow 가 그대로 동작합니다.  
실적으로 FF++ 영상 116개를 16.66GB zip에서, ASVspoof 음성 120개를 464MB parquet에서, DFDC 영상 50개를 96.5GB zip에서 431MB만 받아 뽐냈습니다. Kaggle 계정 없이 크로스도메인 검증을 확보한 것이 이 방식 덕분입니다.

"수치를 재현할 수 있습니까?"

네. 각 수치마다 재현 스크립트가 저장소에 있고 결과는 JSON으로 저장됩니다. 영상은 validate\_detector.py , 음성은 validate\_audio\_spoof.py , 콘텐츠는 validate\_content\_risk.py , 스코어링 결정은 decide\_scoring.py 입니다. 커밋 이력에 측정 시점과 값이 남아 있습니다.

"모델을 직접 학습했습니까?"

안 했습니다. 공개 모델(AASIST, FF++ Xception, faster-whisper, ko-sroberta)을 조합하고 한국어 통화 도메인에서 검증·보정한 것이 저희 기여입니다.  
재칙도 파라미터, 가중치 0.65/0.35, 경계 55/30은 1,050쌍 격자 탐색으로 저희가 적합했습니다.

## 10. 답이 없는 질문 — 이렇게 말할 것

| 질문                      | 답                                                            |
|-------------------------|--------------------------------------------------------------|
| "지연 시간 p95는?"           | 구간별 p95는 안 했습니다. 전체 처리량 기준으로 78초 오디오를 26~29초에 처리한다는 값만 있습니다. |
| "동시 100명이면?"            | 지금 구조로는 안 됩니다. 한 세션만 받습니다. 워커 분리가 다음 단계입니다.                  |
| "모바일 브라우저에서도 실시간이 됩니까?" | 화면이 반응형이라 열리고 파일 업로드는 되지만, <b>모바일 탭 오디오 공유는 확인하지 않았습니다.</b>  |
| "영어나 다른 언어는?"           | 범위 밖입니다. 화법 분류기와 임베딩이 한국어 전용입니다.                             |
| "A/B 테스트나 사용자 검증은?"     | 못 했습니다. 멘토링에서도 지적받았고, 주변 사람 대상 검증이 다음 단계입니다.                 |

기술 질문에서 절대 하지 말 것

- 안 됐는데 숫자를 말하기 — 후속 질문 한 번에 무너진다
- "확장성은 문제없습니다" — 지금은 한 세션이다. 인정하고 방향만 말해라
- "모델을 튜닝했습니다" — 학습은 안 했다. 검증·보정이 정확한 표현이다
- 도메인 밖 한계를 빼고 정확도만 말하기